PRÁCTICA 18

El gimnasio: lazy loading, el build y Zustand

Tópico de Aplicaciones Web · Unidad III · ITSON

Objetivo

Dividir el código de la aplicación por ruta, comparar el resultado del build, cargar imágenes de forma diferida y compartir las inscripciones entre páginas mediante un store de Zustand.

1. Build y carga diferida

Resultado del primer build registrado (código base recibido): Vite 8.3.5, 174 módulos transformados, compilación exitosa. El código base ya genera archivos JavaScript separados por página, por lo que no se debe presentar esta compilación como una versión anterior sin lazy loading.

Archivo	Tamaño (gzip)
index-DRFJ4xNv.js	322.81 kB (105.29 kB)
PaginaClase-DyBASVKG.js	2.39 kB (1.10 kB)
PaginaRegistro-COXUGHRh.js	1.52 kB (0.63 kB)
PaginaClases-DhybVXcC.js	1.46 kB (0.70 kB)
PaginaEntrar-0fxYm4OH.js	1.30 kB (0.62 kB)
PaginaMisInscripciones-DPOtmId-.js	1.04 kB (0.51 kB)
MensajeError-D6AsluaP.js	0.25 kB (0.19 kB)

¿Cuánto se ahorró y por qué el ahorro es pequeño? En una aplicación pequeña, las páginas contienen poco código y la mayor parte del paquete corresponde a dependencias compartidas; lazy loading distribuye la descarga, pero no necesariamente reduce mucho el total.

¿Por qué falla la API si el preview corre en el puerto 4173? Porque localhost:4173 y localhost:5173 son orígenes diferentes; si el backend no permite el origen del preview mediante CORS, el navegador bloquea la petición. Se debe confirmar el error real en la pestaña Red/Consola.

2. Store de inscripciones con Zustand

El store compartido debe mantener las inscripciones, el estado de carga y las funciones para consultar y limpiar los datos. Se llena al iniciar sesión y se vacía al salir. El encabezado muestra el total; el detalle marca horarios inscritos y Mis inscripciones consulta el store.

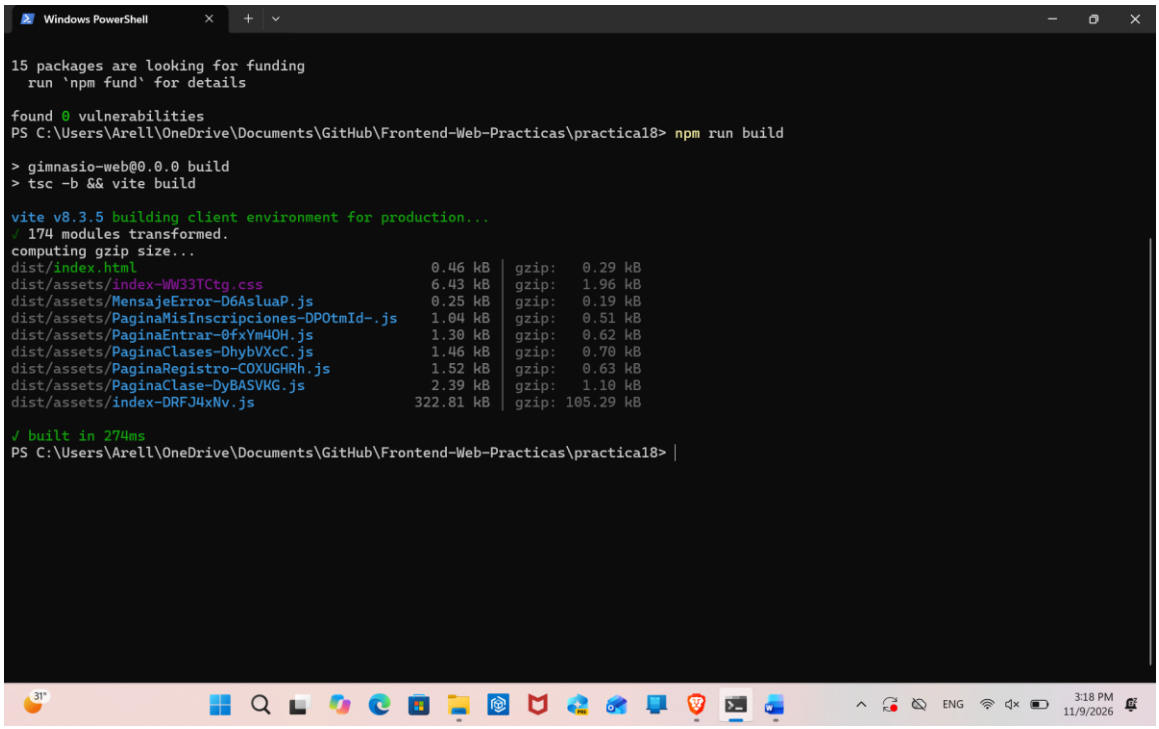
¿Cuándo conviene un contexto y cuándo un store? Context es útil para información transversal relativamente simple, como la sesión o el tema. Un store como Zustand conviene cuando varias pantallas comparten datos que cambian y requieren acciones de actualización, como las inscripciones.

3. Evidencias

Sustituye cada recuadro por la captura correspondiente. Las evidencias deben reflejar pruebas realizadas, no solo el resultado esperado.

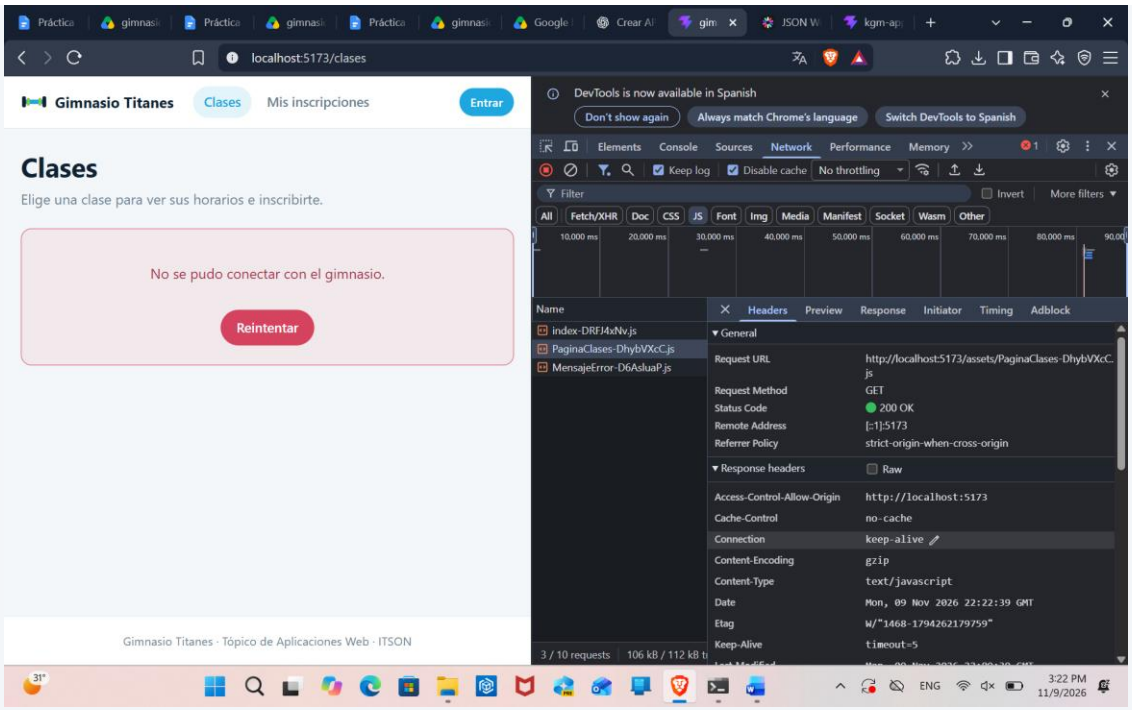
01 — Build inicial

Captura del primer npm run build con archivos y tamaños.



02 — Build comparativo

Captura de la segunda compilación y comparación de tamaños; documentar si el ZIP ya incluía lazy loading.



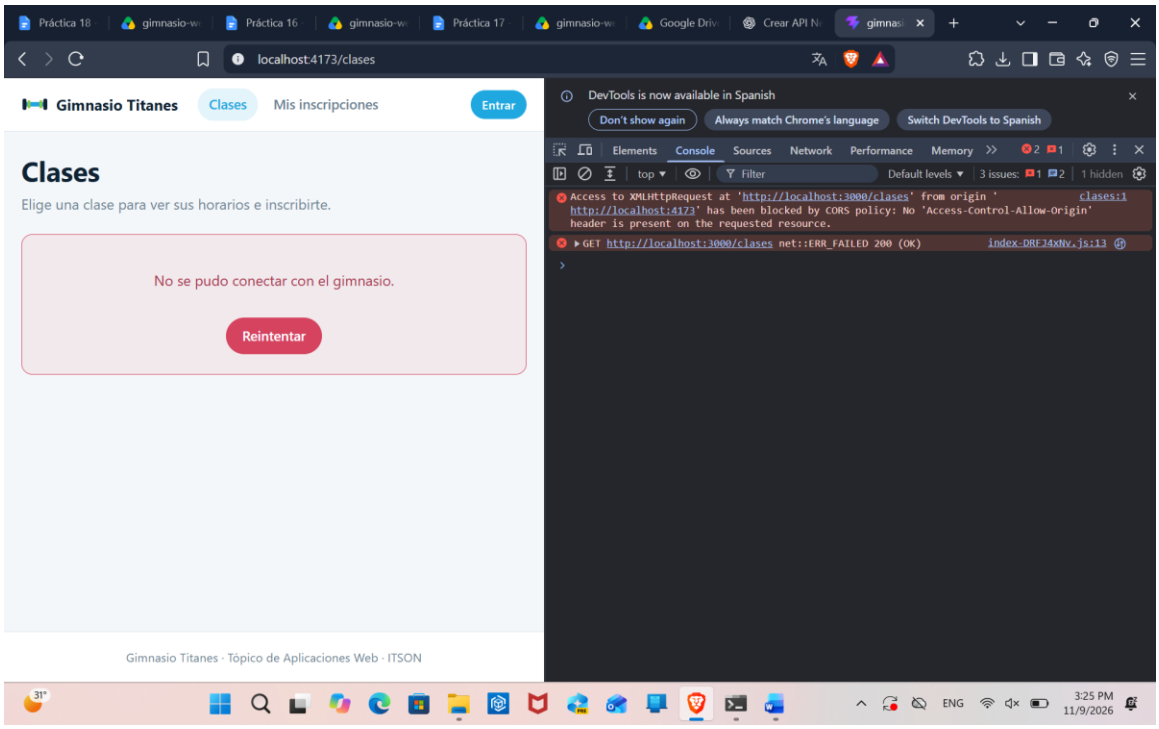
03 — Network: carga de página

Pestaña Red mostrando el archivo de una página descargándose al visitarla.

INSERTAR CAPTURA AQUÍ
03-network-lazy.png

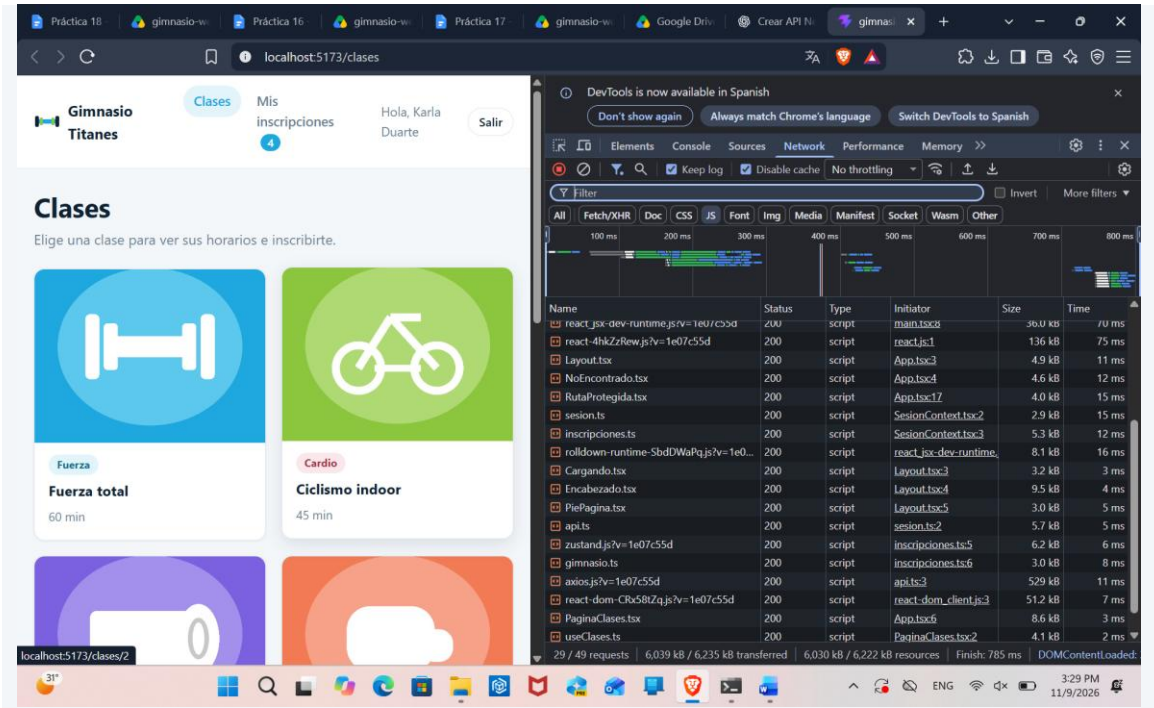
04 — Error de CORS en preview

Pestaña Red o Consola con el fallo observado al usar preview en 4173, si se reproduce.



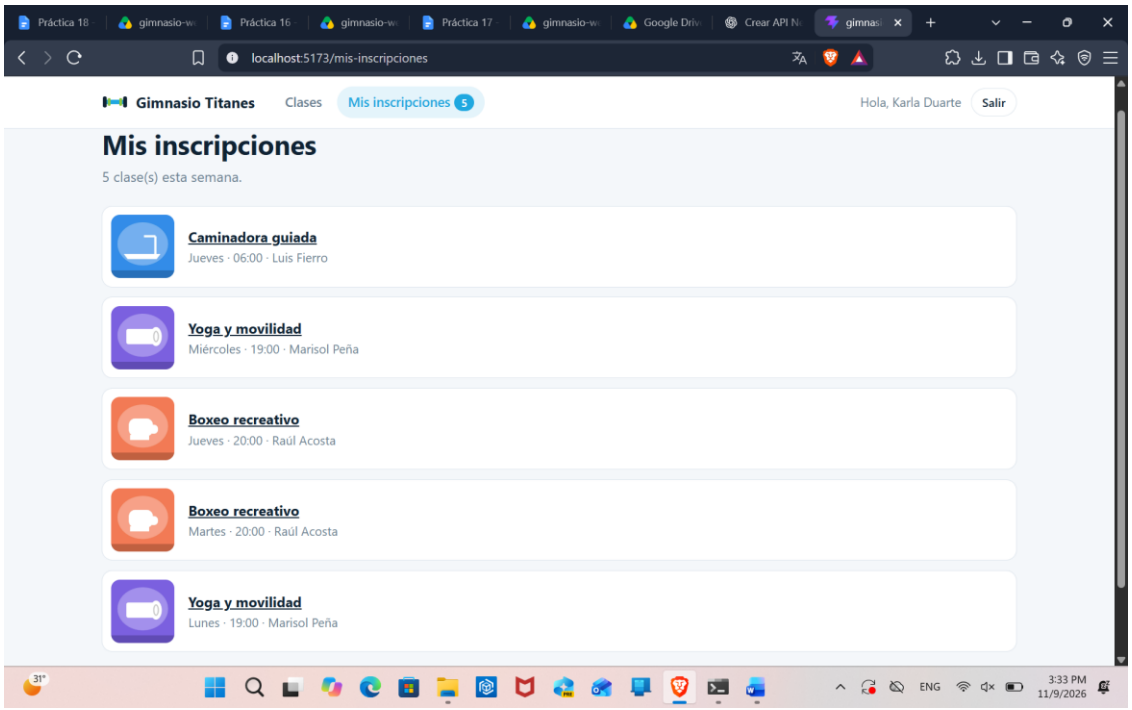
05 — Contador antes

Encabezado con el número de inscripciones antes de inscribirse.



06 — Contador después

Encabezado con el contador actualizado tras inscribirse.



Esta práctica permite comprender cómo la carga diferida distribuye el código de las páginas y cómo un store compartido mantiene sincronizada la información de inscripciones. La comparación del build y las pruebas de Network permiten comprobar los cambios de forma práctica.

5. Entrega

Subir practica18 al repositorio Frontend-Web-Practicas sin node_modules, incluir README.md y carpeta de evidencias. Entregar en iVirtual un PDF con portada oficial de ITSON y enlace al repositorio, nombrado P18_ID.pdf.